

## 計算数学の内容

① アルゴリズム理論入門

② 計算量理論入門

最終目標:  $P \neq NP$  予想の理解.

## 計算の理論

↳ オートマトン, Turing マシンなどの仮想的な機械で処理できるもの.

## 1) アルゴリズム理論

与えられた問題に対してできるだけ早く解く手法を開発.

(厳密 / 近似 / 乱択 / ヒューリスティクス)  
random

## 2) 計算量理論

与えられた問題が「これ以上早くは解けない」ことを示す.

Example:

|             |  |            |  |            |
|-------------|--|------------|--|------------|
| データのサイズ $n$ |  | 素因数分解      |  | 素数判定       |
|             |  | 準指数時間で解ける. |  | 多項式時間で解ける. |
|             |  | or<br>指数   |  |            |

← 分解は超多項式時間のアルゴリズムしか存在しない

↑ よりhard.

Ref

easy 多項式:  $n, n^2, \dots$ ↓ 準指数:  $2^{(\log n)^2}$ hard. 指数:  $2^n, 3^n, (2^{n^2})$ 

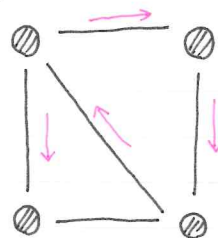
超多項式

⑥ 自然数  $m$  のデータサイズは?  $\log_{10} m$ 

PCにとっては「どれだけメモリを使って表わせるか」.

- パス: 辺を ~~た~~ 辿った有限回の移動
- 単純パス: 同じ頂点を高々1回しか通らない移動

Example.



パスが単純パスではない

- Euler パス: 全ての辺をちょうど1回通るパス
- Hamilton パス: 全ての頂点をちょうど1回通るパス

|          |                       |
|----------|-----------------------|
| Euler パス | Hamilton パス           |
| 多項式時間    | 超多項式時間 (*P ≠ NP 仮定下). |

より hard

\*データサイズは「頂点数」

同じところをクルクルはい

|       |                       |
|-------|-----------------------|
| 最短経路  | 最長(単純)経路              |
| 多項式時間 | 超多項式時間 (*P ≠ NP 仮定下). |

↓ 応用

最適デリバリールート探索

### 3) 計算可能性理論

与えられた問題が有限時間で解けるかを示す.

key: church - Turing Thesis

⑥ どんな問題なら解けないのか?

Diophantine 方程式 (整数係数の多項式 = 0) が与えられたとき、それが整数解を持つか Yes/No で判定せよ.

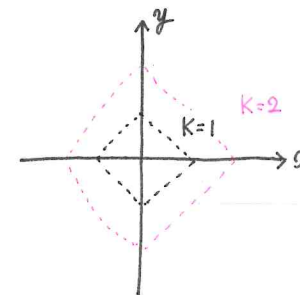
Example:  $xy^2 + 2x^2z + 3yz - 1 = 0$ ,  
 $x^2 + 2y^2 + 1 = 0$  は整数解をもつか?

常に有限時間で  
正しく答えるアルゴリズムは  
存在しない.

Tips: Diophantine 方程式の整数解問題は Hilbert の問題の1つ.

⑥ Diophantine 方程式に整数解が存在することが保証されているとき  
その解を有限時間内で見つけ出せるか?  
⇔ 格子点上の特定の点を見つけて出せるか?

$K=0$  から  $|x| + |y| = K$  となる格子点を  $\pm$  かし、  
解がなければ  $K = K+1$  と更新する.



解が「存在する」と  
保障されているときは o.k.

⑥ 解が「存在するかわからない」とき  
どこまでやれば o.k.?  
→ 一般には無理.

Hilbertの第10問題(変数 ver.).

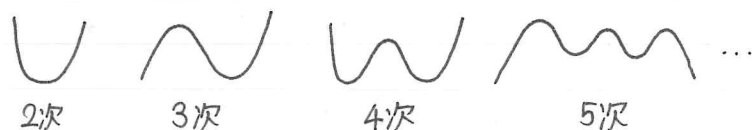
$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

$$(a_i \in \mathbb{Z}, a_n \neq 0)$$

●  $f(x)=0$  の整数解の有無を答えるアルゴリズムは?

→ ある.

方針 1.



アイデア: 絶対値が大きくなれば  $x \in \mathbb{Z}$  ならば  $f(x) \neq 0$ .

$$a := \max |a_i|, |x| \geq 1 \text{ のとき}$$

$$|f(x)| \geq |a_n x^n| - (|a_{n-1} x^{n-1}| + \dots + |a_0|)$$

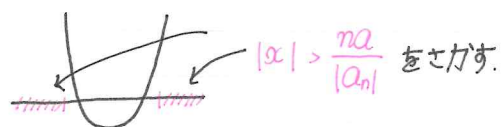
$$\geq |a_n x^n| - (|a_{n-1}| \cdot |x|^{n-1} + \dots + |a_0|)$$

$$\geq |a_n x^n| - n \cdot a \cdot |x|^{n-1}$$

$$= \underbrace{|a_n|}_{\text{IV}} \cdot \underbrace{|x|^{n-1}}_{\text{O}} \cdot \left( |x| - \frac{na}{|a_n|} \right)$$

$$|x| > \frac{na}{|a_n|} \text{ ならば } |f(x)| \neq 0$$

上図を使うと



Algorithm 1

●  $K=0$

●  $f(K)=0$  または  $f(-K)=0$  なら

「整数解あり」と出力し終了.

●  $K=K+1$  として  $\begin{cases} K \leq \frac{na}{|a_n|} \text{ なら 前へもどる.} \\ K \geq \frac{na}{|a_n|} \text{ なら 「整数解なし」と出力して終了.} \end{cases}$

アルゴリズムができた  $\Rightarrow$  その問題は解ける?

次のギモン: 「そのアルゴリズムは高速?」

→ 入力サイズの多項式時間で停止する?

$a_i$  を並べて配列で表現  $\leftarrow$  Standard Encoding  
サイズ  $N := \log |a_0| + \dots + \log |a_n|$  とする.

2進法.

$$1 \text{ 桁: } 0-1 \text{ (0-1)}$$

$$2 \text{ 桁: } 2-3 \text{ (10-11)}$$

$$3 \text{ 桁: } 4_{(100)} - 7_{(111)}$$

⋮

$$k \text{ 桁: } 2^{k-1} \sim 2^k - 1$$

$n$  の桁数は  $\lg n + 1$

Algorithm 1 の (最悪計算) 時間は?

$2na/|a_n|$  回程度  $f(x)$  の値を計算する.

$\hookrightarrow N$  に対して超多項式

Example.  $|a_0| \gg |a_i| \text{ (} i \neq 0 \text{)}$  定数項のみ異常に大きい.

$$\text{ならば } N \approx \log |a_0|$$

方針 2.

方針 1 では  $\mathbb{C}[i] \text{ in } \mathbb{Q}$  でしほり込んだ  
 $\leadsto$  有理根定理を使う.

Thm (有理根定理).

$p, q$  を互いに素な整数とする.

$f(p/q) = 0$  ならば  $p$  は  $a_0$  の約数かつ  $q$  は  $a_n$  の約数.

$f(x)$  の整数解は

- $a_0$  の約数
- 絶対値が  $|a_0|$  以下.

がわかる.

Algorithm 2

-  $|a_0| \leq x \leq |a_0|$  の整数  $x$  に対して  
 $f(x) = 0$  をすべて調べる.

Algorithm 3

①  $a_0$  を素因数分解.

②  $a_0$  の約数  $b$  が  $f(b) = 0$  をみたすか調べる.

最悪計算時間考える.

Alg 2 :  $2|a_0|$  回程度  $f(x)$  の値を計算.

$\hookrightarrow N$  に対して超多項式.

Example.  $|a_0| \gg |a_i|$  ( $i \neq 0$ ) ならば  $N = \log |a_0|$ .

Alg 3 : ① 素因数分解は  $P \neq NP$  とすると  
 超多項式.

②  $a_0$  以下の素数の個数  $\pi(a_0)$  は

$$\left( \lim_{x \rightarrow \infty} \frac{\pi(x)}{x / \log x} = 1 \right)$$

高次方程式を解くときに使う.

3. 3. の利用.

1変数の Hilbert の第10問題を多項式時間で解くアルゴリズムは  
 存在する?

$\rightarrow$  存在する (1982).

key: F. Cucker et al : A Polynomial Time Algorithm  
 for Diophantine Equations in One Variable  
 J. Symbolic Computation 27, pp21-29. 1999.

多項式の表現: Sparse Encoding  
 (素).

$f(x)$  に対して  $\{(i, a_i) : a_i \neq 0\}$  で表す.

②  $f(x) = x^n$  の場合

- SE のサイズ
- Sparse Encoding のサイズ

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

$$(a_i \in \mathbb{Z}, a_n \neq 0)$$

② St E :  $f(x)$  を配列  $(a_0, a_1, \dots, a_n)$  で表現.  
 $\Rightarrow$  サイズ:  $\log a_0 + \log a_1 + \dots + \log a_n$ .

② Sp E :  $f(x)$  を  $\{(i, a_i) : a_i \neq 0\}$  で表現.

Example:  $x^n$  のサイズは?

$$\text{St E: } (0, 0, 0, \dots, 1) \Rightarrow \text{サイズ: } \log n + 1 = O(n)$$

$$\text{Sp E: } \{(n, 1)\} \Rightarrow \text{サイズ: } (\log n) + 1 = O(\log n)$$

機械学習に有効.



べき乗計算(多項式の値の計算).

$f(x) = x^n$  と  $b$  が与えられて  $f(b) = b^n$  を求める.

⑥ 計算時間は?

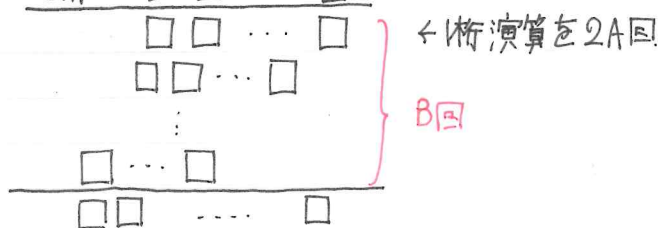
- St E: 入力サイズ:  $O(n + \log b)$ .

1回の計算時間

A桁とB桁の整数の乗算時間:  $O(\frac{AB}{2})$  回の1桁演算.

A桁:  $\square \square \dots \square$

B桁:  $\square \square \dots \square$



1回の乗算で計算を行う数値の桁数は高々  $n \log b$  ( $b^n$  の桁数)

乗算の回数: 自明なとき:  $n$  回

doubling:  $\log n$  回

合計で多項式時間

- Sp E: 入力サイズ:  $O(\log n + \log b)$ .

計算結果は  $b^n$  なので答えを書き込むだけで超 polynomial time.

サイズ:  $n \log b$

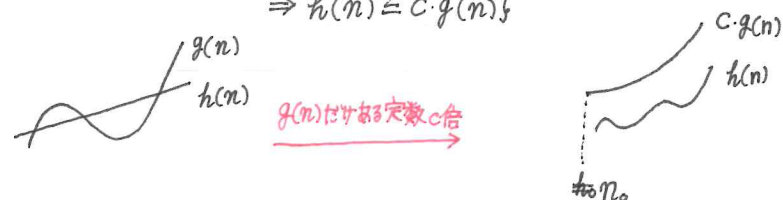
漸近記法: Landau の  $O$

関数  $f, g, h: \mathbb{Z}_{>0} \rightarrow \mathbb{R}_{>0}$  とする.

Def (big-O).

$O(g(n)) = \{h(n); \exists c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{Z}_{>0}, n \geq n_0$

$\Rightarrow h(n) \leq c \cdot g(n)\}$



Example:  $2n+1 \in O(n^2)$ ?

→ Yes ( $c=1, n_0=3$  のとき  $n^2 - (2n+1) \geq 0$ ).

Example:  $2n^2 \in O(n^2)$ ?

→ Yes ( $c=2, n_0=1$  のとき  $2 \cdot n^2 - 2n^2 \geq 0$ ).

Example:  $n^3 \in O(n^2)$ ?

→ No. ( $\forall c \in \mathbb{R}_{>0}, n \geq c$  のとき  $n^3 - c \cdot n^2 \geq 0$ ).

$f(n) = O(g(n))$  は  $f(n) \in O(g(n))$  の1.

Example:  $k \in \mathbb{R}_{>0}, d \in \mathbb{R}_{>1}$  のとき

$n^k = O(d^n)$  を示せ ← polynomial < exponential.

Answer:  $k' := \lceil k \rceil$

←  $\mathbb{Z}[x] := \min\{a \in \mathbb{Z}; a \geq x\}$

$$e^x = 1 + x + \frac{1}{2!}x^2 + \frac{1}{3!}x^3 + \dots + \frac{1}{k!}x^k$$

$$x \geq 0 \text{ ならば } e^x \geq \frac{1}{k!}x^k$$

$$x < 0 \text{ ならば } x = n \log d \ (n \in \mathbb{Z}_{>0}) \text{ のとき}$$

$$e^x = e^{n \log d} = d^n$$

$$\frac{1}{k!}x^k = \frac{1}{k!}(n \log d)^k = \frac{(\log d)^{k'}}{k!}n^{k'} \geq n^k$$

つまり  $d^n \geq n^k$  が成り立つ.

$$C = ((\log d)^{k'}/k!)^{-1}, n_0 = 1 \text{ とすると}$$

$$n \geq n_0 \text{ ならば } n^k \leq C \cdot d^n \text{ なので } n^k = O(d^n).$$

Example:  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \in [0, \infty)$  ならば  $f(n) = O(g(n))$ .

Remark: 逆は成り立たない.

$$CE: f(n) = \begin{cases} 1 & (n: \text{偶数}) \\ n & (n: \text{奇数}) \end{cases}, g(n) = n$$

Answer:  $d := \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  とする.  $\varepsilon = 1/2$  に対して  $N \in \mathbb{Z}_{>0}$ ,

$$n \geq N \text{ ならば } |d - f(n)/g(n)| \leq 1/2$$

$$\text{つまり } -1/2 \leq d - f(n)/g(n) \leq 1/2$$

$$\text{よって } f(n)/g(n) \leq d + 1/2, f(n) = (d + 1/2) \cdot g(n)$$

$$\text{以上より } C = d + 1/2, n_0 = N \text{ とすると } f(n) = O(g(n)).$$

Example (Stirlingの公式).

$$\lim_{n \rightarrow \infty} \frac{n! \cdot e^n}{\sqrt{2\pi n} \cdot n^n} = 1$$

$$n! = O\left(\frac{n^{n+\frac{1}{2}}}{e^n}\right), \quad \frac{n^{n+\frac{1}{2}}}{e^n} = O(n!)$$

small-o 記法.

$$\Omega(g(n)) = \{h(n); \forall c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{Z}_{>0}, n \geq n_0 \text{ ならば } f(n) \leq c h(n)\}$$

\*  $h(n) \in O(g(n))$  を  $f(n) = \Omega(g(n))$  と表す.

Example:  $2n \in O(n^2)$  は成立?  $\rightarrow$  成立.

$$cn^2 - 2n = cn(n - \frac{2}{c}) \text{ ならば } n \geq \frac{2}{c} \text{ で } 2n \leq cn^2$$

Example:  $2n \in \Omega(n^2)$  は成立?  $\rightarrow$  不成立

$$c=1 \text{ のとき } \forall n_0 \in \mathbb{Z}_{>0}, n \geq n_0 \text{ で } 2n^2 \geq 1 \cdot n^2$$

Exercise: 以下は同値であることを示せ.

i)  $f(n) = \Omega(g(n))$ .

ii)  $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$ . 仮定:  $f, g: \mathbb{Z}_{>0} \rightarrow \mathbb{R}_{>0}$ .

定数  $c > 0$  に対して  $f(n) \leq cg(n) \Leftrightarrow f(n)/g(n) < c$ .

i)  $\forall c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{Z}_{>0}, n \geq n_0 \text{ ならば } f(n) \leq c \cdot g(n)$ .

ii)  $\forall c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{Z}_{>0}, n \geq n_0 \text{ ならば } f(n)/g(n) \leq c$ .

Example: 定数  $k \in \mathbb{R}_{\geq 0}, d \in \mathbb{R}_{>1}$  に対し  $n^k = O(d^n)$  を示せ.

Hint:  $\Omega$  のときと同様に Taylor 展開.

Answer:  $k' = \lceil k \rceil$  とする.

$x \geq 0$  では  $e^x \geq 1/(k'+1)! \cdot x^{k'+1}$  であるから

$$\frac{(k'+1)!}{x} \geq \frac{x^{k'}}{e^x}$$

$x < 1$  に  $x = n \log d$  ( $n \in \mathbb{Z}_{>0}$ ) としたとき

$$(\text{左辺}) = \frac{(k'+1)!}{x}$$

$$(\text{右辺}) = \frac{(n \log d)^{k'}}{d^n} \geq \frac{n^{k'}}{d^n} (\log d)^{k'}$$

つまり

$$\frac{(k'+1)!}{(\log d)^{k'+1}} \cdot \frac{1}{n} \geq \frac{n^{k'}}{d^n}$$

なので

$$\lim_{n \rightarrow \infty} \frac{n^{k'}}{d^n} = 0. \quad \therefore n^{k'} = O(d^n) \quad \square$$

その他の Notation

①  $\Omega(g(n)) = \{h(n); \exists c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{Z}_{>0}, n \geq n_0 \text{ ならば } c \cdot g(n) \leq h(n)\}$ .

②  $\omega(g(n)) = \{h(n); \forall c \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{Z}_{>0}, n \geq n_0 \text{ ならば } c \cdot g(n) \leq h(n)\}$ .

③  $\Theta(g(n)) = \{h(n); \exists C_1, C_2 \in \mathbb{R}_{>0}, \exists n_0 \in \mathbb{Z}_{>0} \text{ ならば } C_1 g(n) \leq h(n) \leq C_2 g(n)\}$ .

④  $\tilde{O}_1(g(n)) = \{h(n); \exists k \in \mathbb{Z}, h(n) = O(g(n) \cdot (\log n)^k)\}$ .

⑤  $\tilde{O}_2(g(n)) = \{h(n); \exists k \in \mathbb{Z}, h(n) = O(g(n) \cdot \frac{n^k}{\log(g(n))})\}$ .

↓ 書き直し.

⑥  $\tilde{O}_2(g(n)) = \{h(n); \exists k \in \mathbb{Z}, h(n) = O(g(n) \cdot \{\log g(n)\}^k)\}$ .

⑦  $O^+(g(n)) = \{h(n); \exists k \in \mathbb{Z}, h(n) = O(g(n) \cdot n^k)\}$ .

レポート課題 (漸近記法).

1.  $f(n) = O(g(n))$  かつ  $g(n) = \Theta(n)$  ならば  
 $f(n) = O(n)$ .

2. 以下が同値であることを示せ.

i)  $f(n) = \Theta(g(n))$

ii)  $f(n) = O(g(n))$  かつ  $f(n) = \Omega(g(n))$ .

3. 次を示せ.

i)  $n^2 \cdot 2^n \neq \tilde{O}(2^n)$

ii)  $n^2 \cdot 2^n = \tilde{O}_2(2^n)$ .

4.  $n! = \Omega(n^n)$  を示せ. 期限 ~ 7/20?

ソート

⑤ INPUT: 長さ  $n$  の整数配列.

$\{a_i\}_{i=1,2,\dots,n}$

OUTPUT: 入力を昇順に並べ替えた配列.

Algorithm 1. 選択ソート

Step 1.  $k := 1$ .

Step 2.  $l = k, \dots, n$  に対して順番に  $a_k$  と  $a_l$  を  
 比較する.  $a_k < a_l$  ならば並べ替える.

Step 3.  $k = l - 1$  ならば終了,  $k \neq l - 1$  ならば  $k \rightarrow k + 1$  と  
 更新し, Step 2 を繰り返す.

Example: 列 3241 を選択ソートする.

$k=1$  3 2 4 1  $\rightarrow$  2 3 4 1

$\rightarrow$  2 3 4 1  $\rightarrow$  1 3 4 2

$k=2$  1 3 4 2  $\rightarrow$  1 2 4 3

$k=3$  1 2 3 4 END.

計算時間について考える.

比較回数を計算時間として評価する.

Step 2 ~~for~~  $k=1$  のとき  $n-1$  回

で  $k=2$  :  $n-2$  回

$\vdots$

$k=n-1$  : 1 回

$\rightarrow$  計:  $n(n-1)/2$  回 ( $\Theta(n^2)$  回).

Algorithm 2: マージソート

マージ操作: 昇順に並んだ 2 つの整数  $\{a_i\}, \{b_i\}$  に対し,  
 2 つの配列を先頭から順に比較し,  
 小さい方を取り出して並べる.

Example 列 156, 234 を並べ替える.

234 156  $\rightarrow$  234 56

1 2

$\rightarrow$  34 56 4 56

1 2 3 1 2 3 4

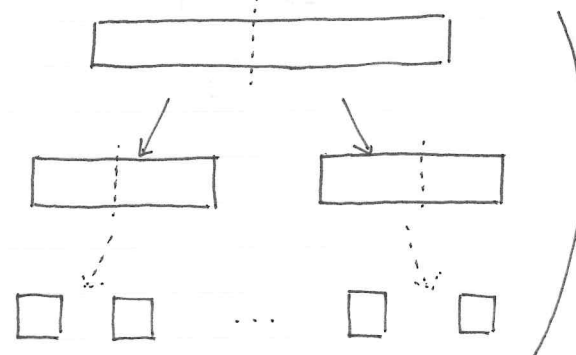
$\rightarrow$  56

1 2 3 4 56 END.

比較の回数は高々  $n_1 + n_2$  回 ( $\approx O(n)$  回).



Step 1. 配列を長さ1になるまで分割.



ソート済配列 (長さ1).

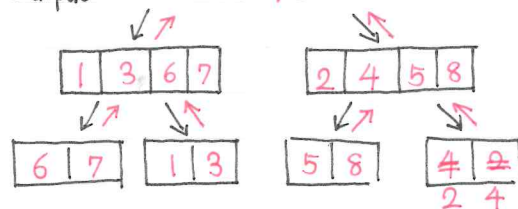
段数  $n$   
1段の処理に  
 $n$ 回の比較

Step 2. 下から順にマージ操作を行う.

Example:

Input 7 6 13 8 5 4 2

Output 1 2 3 4 5 6 7 8



分割統治法  
divide and conquer

計算時間 (比較の回数).

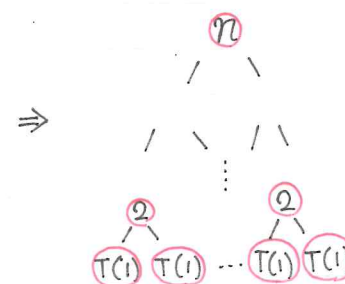
配列長さ  $n$  のとき 計算時間  $T(n)$

①  $n = 2^k$  のとき

$$T(n) \leq 2 \cdot T\left(\frac{n}{2}\right) + n \leq 2 \left( 2 T\left(\frac{n}{4}\right) + \frac{n}{4} \cdot 2 \right) + n \leq \dots \leq T(1).$$

$$\begin{aligned} & \stackrel{\parallel}{=} 2^2 T\left(\frac{n}{4}\right) + 2n \\ & \leq 2^k T(1) + kn. \end{aligned}$$

$$T(n) \Rightarrow \begin{matrix} n \\ / \quad \backslash \\ T(n/2) \quad T(n/2) \end{matrix} \Rightarrow \begin{matrix} n & & n \\ / \quad \backslash & & / \quad \backslash \\ T(n/4) & T(n/4) & T(n/4) & T(n/4) \end{matrix}$$



再帰木.

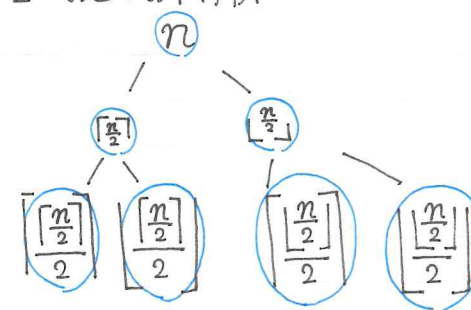
$$\begin{aligned} T(1) &= O. \\ T(n) &\leq k \cdot n = n \lg n \\ \therefore T(n) &= O(n \lg n) \end{aligned}$$

②  $n \neq 2^k$  のとき

①で長さ  $m$  に分割するとき  $\lceil m/2 \rceil$  と  $\lfloor m/2 \rfloor$  の配列に分割.

②のマージ操作は長さ  $\lceil m/2 \rceil$  と  $\lfloor m/2 \rfloor$  に対して行うので  
比較回数は高々  $m$  回.

$2^{k-1} < n < 2^k$  のときの再帰木



下の木は上の木の一部.

対応する頂点は上と下になっている.

$T(n) \leq \text{○の総和} \leq \text{○の総和}$

$$\stackrel{\parallel}{=} 2^k \cdot k$$

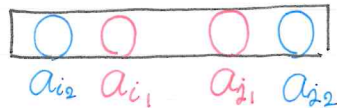
$\wedge$

$$2n \lg 2n = O(n \lg n)$$



Quiz: より高速なアルゴリズムは存在する?

- ① 任意のアルゴリズムを1つ固定.
- ② アルゴリズムの挙動を木で表現.  
(比較の結果で分岐).



$\langle a_{i1} \geq a_{i2} \text{ を比較} \rangle$

1回目

$a_{i1} \geq a_{i2}$

$a_{i1} < a_{i2}$

$\langle a_{i2} \geq a_{i3} \text{ を比較} \rangle$

$\langle a_{i3} \geq a_{i4} \text{ を比較} \rangle$

2回目

$\langle \text{葉} \rangle$

途中でノードが終了

$\langle \text{葉} \rangle$

$\langle \text{葉} \rangle$

k回目

比較回数の最大値

● 葉の個数は高々  $2^k$

● 配列の大小関係の個数:  $n!$

$$\Rightarrow 2^k \geq n!$$

Stirling の公式より十分大きな  $n$  で

$$\frac{n!}{\sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n} \geq \frac{1}{2}$$

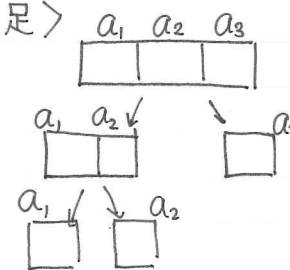
$$(2^k >) n! \geq \frac{1}{2} \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

↓ log をとる.

$$\log 2^k = k, \quad \log n! = \underbrace{n \log n}_{\text{大}} + \log \left( \frac{1}{2} \sqrt{2\pi} \right)^{\frac{1}{2}} - n \log e$$

$$\therefore k = \Omega(n \log n).$$

<補足>



マージソートの分割過程.

Input: 1: 1 2 3

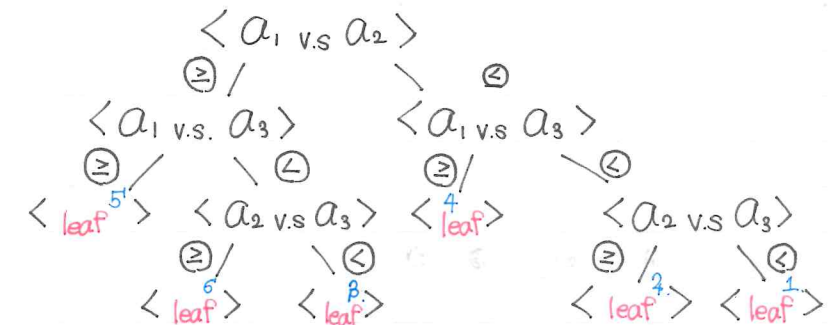
2: 1 3 2

3: 2 1 3

4: 2 3 1

5: 3 1 2

6: 3 2 1



Algorithm: バケットソート ( $O(n)$ ?)

Step 1: 配列の中の最大値  $m$  をさがす.

Step 2:  $m$  個の変数  $x_1, x_2, \dots, x_m$  を 0 で初期化.

Step 3:  $k=1, \dots, n$  に対して  $x_{a_k}$  を +1 する.

Step 4:  $l=1, \dots, m$  に対して 1 から順に  $x_l$  の値の個数だけ  $l$  を並べる.

Example: 4, 2, 1, 4, 5, 8, 7

Step 1: 最大値  $m=8$ .

Step 2:  $x_1, \dots, x_8$  を用意 (中身はすべて 0).

$(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8)$   
 $(0, 0, 0, 0, 0, 0, 0, 0)$

Step 3:  $x_4, x_2, x_1, x_4, x_5, x_8, x_7$  を +1

$(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8)$   
 $(1, 2, 0, 4, 5, 0, 7, 8)$

Step 4: 1, 2, 4, 4, 5, 7, 8

<計算時間>

Step 1, 3  $\Rightarrow O(n)$  時間

Step 2, 4  $\Rightarrow O(m)$  時間

$m = O(2^n)$  なら  $O(2^n)$  時間かかる.

⑥ サブルーチンと計算の難しさ.

区間スケジューリング

Input:  $n$  個の閉区間  $I_i = [a_i, b_i]$  の集合  $S$ .

( $i=1, \dots, n, a_i, b_i \in \mathbb{Z}$ ).

Output:  $T \subset S$  で次の条件を満たす最大サイズのものの  $l$ .  
 (条件)  $T$  中の閉区間は共通部分をもたない.

意味.

- 施設の利用申込を考える.
- 申込には開始/終了を書き込む.
- 同時に 1 組しか利用できない.

ナイーブな Algorithm.

$S$  の sub set を 100 調べ, 条件を満たす subset のうち最大サイズのものを見つける.

$\rightarrow$  hyper poly. time

ソートを使ったアルゴリズム(\*)

Step 1:  $b_i$  が小さい順に区間を並べかえる.

並べかえたものを先頭から  $J_1, J_2, \dots, J_k = [c_k, d_k], \dots, J_n$  とする).

Step 2:  $T = \{J_1\}, k=2, D = d_1$

$\uparrow$  output の候補

$\uparrow$   $T$  の区間で右端の値

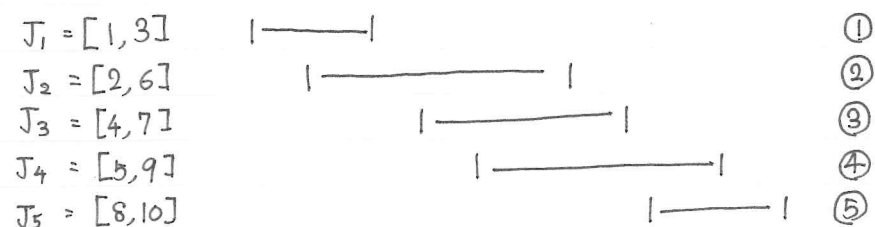
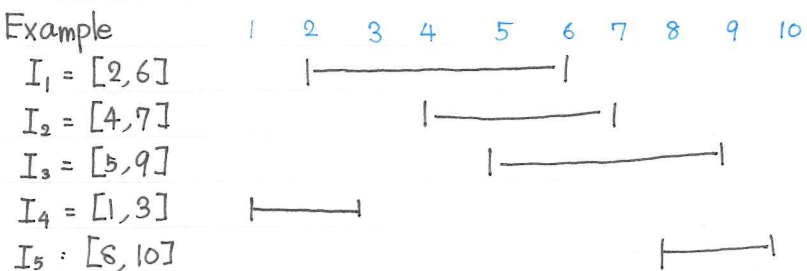
Step 3: もし  $J_k = [c_k, d_k]$  が  $D < c_k$  ならば

$J_k$  を  $T$  に加えて  $D := d_k$

Step 4:  $k = n$  なら終了

$k \neq n$  なら  $k$  を +1 して Step 3 へ

Example



- ①  $T = \{J_1\}$ ,  $D = 3$
- ②  $T = \{J_1\}$ ,  $D = 3$
- ③  $T = \{J_1, J_3\}$ ,  $D = 7$
- ④  $T = \{J_1, J_3\}$ ,  $D = 7$
- ⑤  $T = \{J_1, J_3, J_5\}$ ,  $D = 10$

正當性

(\*) の出力が最大サイズではないと仮定して矛盾を導く.

(\*) で得られた区間を  $K_1, K_2, \dots, K_p$  $(K_i = [e_i, f_i] \text{ で } e_1 \leq f_1 < e_2 \leq f_2 < \dots < e_p \leq f_p)$ 

条件を満たす最大サイズの区間集合を

 $L_1, L_2, \dots, L_q$  ( $p < q$ : 仮定より)

とする.

 $(L_i = [g_i, h_i] \text{ で } g_1 \leq h_1 < g_2 \leq h_2 < \dots < g_q \leq h_q)$ まず  $f_i \leq h_i$  ( $i = 1, \dots, p$ ) を帰納的に示す.(base) (\*) の Step 1. で右端で区間をソートしているので  $f_1 \leq h_1$ (step)  $f_i \leq h_i$  を仮定して  $f_{i+1} \leq h_{i+1}$  を示す.       が成立しないと仮定し、背理法を使う.